

DBT Day 1

Shivam Zanjurne · Thu, 08 Oct 26 · MySQL setup, data fundamentals, DBMS vs file system, SQL categories + first queries

Table of Contents

1. MySQL Installation and Environment Setup
2. Three Ways to Access MySQL
3. Data, Information, Knowledge, Wisdom
4. Types of Data
5. Why DBMS and RDBMS Are Needed
6. File System vs RDBMS
7. Primary Key, Foreign Key and Relationships
8. CHAR vs VARCHAR
9. Indexes
10. Data Models: Conceptual, Logical, Physical
11. Types of DBMS
12. SQL Language Categories
13. All SQL Commands from This Session

MySQL Installation and Environment Setup

Full MySQL installation on Windows using the official MySQL Installer, then PATH setup so `mysql` works from any CMD window.

Installation steps (Windows)

1. Download the **MySQL Installer** from the official MySQL website ($\approx$ **432 MB**).
2. Run the installer and choose **Custom** installation.
3. Expand **MySQL Server** and move the latest **MySQL Server 8.0.x** to the right panel (green arrow).
4. Expand **Applications** and add **MySQL Workbench only** — nothing from Connectors.
5. Click **Next**, then **Execute** to install.
6. Keep the default port **3306** (change only if something else already uses it).
7. Set a **root password** you will remember — recovery is painful if you forget it. Localhost password strength matters less than memorability.
8. Do **not** add extra users during setup.
9. Finish — Workbench and the MySQL Command Line Client may open automatically.

Service	Default port
MySQL	3306
PostgreSQL	5432

Adding MySQL to the system PATH (Windows)

So that `mysql` works from any CMD window:

1. Go to `C:\Program Files\MySQL\MySQL Server 8.0\bin`
2. Copy that full path
3. **Environment Variables** → System Variables → **Path** → **New** → paste → OK
4. Open a **new** CMD window and verify:

```
mysql --version
```

Example output:

```
mysql Ver 8.0.31 Distrib 8.0.31, for Win64 (x86_64)
```

Any **8.0+** version is fine for this module.

Three Ways to Access MySQL

All three run the **same SQL**. Only the interface differs. Exams and this module use **CMD**.

1. CMD (Command Prompt) — primary

```
mysql -u root -p
```

Enter the root password → you get the `mysql>` prompt.

2. MySQL Workbench (GUI)

Visual table design and query editor. Avoid for learning: auto-suggest / auto-complete stop you from writing queries yourself.

3. MySQL Shell

Advanced shell with three modes:

```
\sql      -- SQL mode
\py       -- Python mode
\js       -- JavaScript mode
```

Connect:

```
\connect root@localhost:3306
```

Not required for this module — CMD is enough.

Data, Information, Knowledge, Wisdom

What is data?

Data is a collection of **raw values with no meaning or context** on their own.

If you see `20`, `ABC`, or a random string with no labels, you cannot tell what they represent — that is raw data.

What is information?

When data is mapped to a **structure or label**, it becomes **information**.

Field	Value
PRN	2026040520200
Name	Aniket
Email	aniket@gmail.com
Age	20

Now `20` means something — it is tied to the label **Age**. Structure + context = information.

What is knowledge?

Knowledge is what you get when you **analyse information** and use it to decide or conclude.

Hospital example:

- Doctor collects structured data (name, age, blood group, BP) → **information**
- Doctor analyses it and prescribes medicine → **knowledge**
- On the next visit, compares old vs new reports and adjusts treatment → knowledge in action

What is wisdom?

Wisdom is long-term direction that comes from applying knowledge repeatedly.

After treating many similar patients and seeing outcomes, the doctor knows which treatments work for which profiles — that accumulated insight is **wisdom**.

Business and real-world examples

- **Product launch:** Sales data from Mumbai and Kolhapur → regional knowledge → decide whether to launch in Gujarat (wisdom).

- **Weather prediction:** Years of temperature / humidity data → models that predict floods (knowledge from structured information).
- **Modern business:** Retail, finance, healthcare, logistics all run on analysed data for decisions.

DATA (raw) —structure→ INFORMATION (labelled) —analysis→ KNOWLEDGE (decision) —experience→ WISDOM (vision)

Types of Data

Structured data

Predefined format, stored in **rows and columns** (spreadsheet / relational table). Every record follows the same schema.

Example: student table with fixed columns `id`, `name`, `age`, `email`.

Unstructured data

No predefined format:

- Images
- Audio
- Video
- Free-form text documents

Cannot be stored directly as a flat table.

Semi-structured data

Organised, but **not** strict rows-and-columns. Classic example: **JSON** (key-value / nested objects).

```
{
  "library": {
    "name": "Central Library",
    "location": "Mumbai",
    "books": [
      { "title": "Data Structures", "author": "Aniket" }
    ],
    "members": [
      { "name": "Yash", "email": "yash@gmail.com" }
    ]
  }
}
```

You can navigate to `library.name`, but it is not a flat table. Usually stored in **NoSQL** (e.g. MongoDB).

Can JSON be stored in MySQL?

Yes. MySQL has a `JSON` data type (demo later with `books_management`). It is for awareness only — not the main style of this module.

Type	Format	Typical store
Structured	Rows & columns, fixed schema	RDBMS (MySQL, PostgreSQL)
Unstructured	No schema (media, free text)	File / object storage, NoSQL
Semi-structured	Organised but flexible (JSON)	MongoDB; also MySQL <code>JSON</code>

Why DBMS and RDBMS Are Needed

The file system approach (traditional)

Before databases, organisations used a **hierarchical file system**: folders within folders, flat files at the leaves.

```
Course/
├── PGCP-AC/
│   ├── student_info.file    ← enrolled students
│   ├── modules.file        ← modules taught
│   └── result.file          ← student results
├── PGCP-BDA/
│   ├── student_info.file
│   ├── modules.file
│   └── result.file
└── PGCP-AI/
    ├── student_info.file
    ├── modules.file
    └── result.file
```

One root → child folders per course → leaf files. The OS file explorer is itself a living tree.

Drawbacks of the file system

1. Data redundancy

Same data stored in multiple files (e.g. name / PRN in both `student_info.file` and `result.file`). Wastes storage and makes maintenance harder.

The same data may be stored in multiple files.

2. Data inconsistency

Update in one file does not update others. DOB changed in `student_info.file` but not in `result.file` → conflicting values for the same student.

If data is updated in one file but not in related files, different files may contain different values for the same field.

3. Difficulty in data access

No built-in query language. Need a custom script for every retrieval (attendance + result + personal details → open many files and combine them yourself).

Retrieving required information may require writing a new program or script for each specific data need.

4. Data isolation

Data scattered across files and formats. A student in two courses may exist in multiple places with no easy link — hard to get one unified view.

Data is scattered across different files and formats, making it difficult to combine and process.

5. Integrity problems

Enforcing rules across files is painful. "Top 3 in PGCP-AC" means: open `result.file`, find top PRNs, then cross-check `student_info.file` for names — error-prone.

In RDBMS, one query:

```
SELECT student.name, student.prn, result.marks, result.rank
FROM student, result
WHERE student.prn = result.prn
      AND result.rank ≤ 3;
```

Maintaining and enforcing data rules across multiple files is difficult and error-prone.

6. Security problems

File systems have only **OS-level** security. Unlocked laptop → all files open. No fine-grained "read-only for this person".

RDBMS: users + `GRANT` / `REVOKE` (e.g. `SELECT` only, no `INSERT` / `UPDATE` / `DELETE`).

File systems offer only OS-level security; RDBMS provides command-level and user-level security via `GRANT` and `REVOKE`.

7. Concurrent access / file-level locking

One user editing a file **locks the whole file**. Others wait. In RDBMS, typically **row-level locking** — others can work on other rows.

In file systems, the entire file is locked during access. In RDBMS, only the specific row is locked (row-level locking).

8. Atomicity problems

Crash mid-write → partial / corrupt data. File system has no automatic rollback. RDBMS: **COMMIT / ROLLBACK / SAVEPOINT** — all or nothing.

Partial data writes in a file system can corrupt data. RDBMS provides rollback and savepoint mechanisms to maintain consistency.

Real-world motivation

Scenario	Why RDBMS matters
Big Billion Day (Flipkart)	~10 lakh users hit the same listings/orders — concurrent reads/writes, no overselling, consistent inventory
Instagram	Authenticate you and show <i>your</i> feed while millions do the same — map session → user data in microseconds
Bank transfer	₹10,000 debited but not credited on crash → RDBMS rolls back the whole transaction (atomicity)

File System vs RDBMS

Terminology

Concept	File system term	RDBMS term
Container	File	Table / Entity / Relation
Column header	Field	Column / Attribute
One data entry	Record	Row / Tuple

Relationships between data containers

- **File system:** Join two files yourself with a custom script.

- **RDBMS:** Relationships via **primary key** and **foreign key**, defined when tables are created — no extra glue code to navigate them.

Programming effort

- **File system:** Custom program for every retrieval → more code, more maintenance.
- **RDBMS:** SQL — one query can filter, join, and sort across tables in milliseconds.

Software development time

- **File system:** Slower — every operation needs custom code.
- **RDBMS:** Faster — queries, stored procedures, and functions; multiple statements can be packaged as one unit.

Network traffic

- **File system:** Often ship the **entire file** to the client for processing → higher traffic.
- **RDBMS:** Process on the **DB server**; send only the **result set** back → lower traffic.

Speed and cost

- **File system:** Slower long-term; duplication raises storage cost.
- **RDBMS:** Faster retrieval (especially with indexes); cheaper day-to-day. Initial licence cost (e.g. Oracle) can be higher than a simple file setup.

Client-side vs server-side processing

```
Client (Browser / App)
  | HTTP request
  ▼
Middleware (e.g. Java + React on server)
  | SQL query
  ▼
DB Server (MySQL / PostgreSQL)
  | result set
  ▼
Middleware → response → Client
```

The client never touches raw tables — only the answer to the query.

Locking and multi-user support

Aspect	File system	RDBMS
Locking	Entire file	Specific row (usually)
Multi-user	Poor concurrency	Built for thousands of connections
Security	OS password only	User + password + <code>GRANT</code> / <code>REVOKE</code> (DB or table level)

Example of command-level security: user `aniket` may only `SELECT` on `student` — no `INSERT`, `UPDATE`, `DELETE`, `DROP`, `CREATE`.

Primary Key, Foreign Key and Relationships

- **Primary key (PK):** column (or set of columns) that **uniquely identifies** each row. No two rows share the same PK value.
- **Foreign key (FK):** column in one table that **references the PK** of another table — how relationships are maintained.

Rule: a foreign key must reference a column that is a primary key in another table.

Example: `student` and `result`



(In class, `prn` was shown linking the two tables; the join on the common key is what matters.)

Retrieve Ritik's details including result:

```
SELECT student.id, student.name, student.prn, result.marks, result.rank
FROM student, result
WHERE student.prn = result.prn
AND student.name = 'Ritik';
```

Joins on `prn`, filters by name. File system would need a custom program; RDBMS does it in one query.

CHAR vs VARCHAR

Both store strings; they differ in **how storage is allocated**.

	CHAR	VARCHAR
Length	Fixed	Variable
Use when	Length is always the same	Length varies
Example	PIN code → <code>CHAR(6)</code>	Name, email → <code>VARCHAR(50)</code>
Storage	Always allocates full declared size	Allocates only what is needed

Rule of thumb:

- Use **CHAR** for fixed length (PIN, gender code, country code).
- Use **VARCHAR** for varying length (names, emails, cities).

Both need a size: `CHAR(6)` , `VARCHAR(30)` , etc.

Indexes

An **index** speeds up queries on a column by finding rows **without a full table scan**.

CDAC registration / seat allocation example

1. Student registers on the CDAC portal (MySQL) — form number, UID, email, payment stored.
2. Seat allocation system (separate project, often PostgreSQL) must verify in **microseconds**: registered? paid? seat already allocated?
3. Millions may log in at once — scanning every registration row is too slow.
4. An **index on form number** jumps straight to the row.

Indexes are critical for high-traffic, time-sensitive lookups.

Data Models: Conceptual, Logical, Physical

Design moves through three levels. Running example: **Library Management System**.

Conceptual data model

High-level: **what entities exist** and how they relate — not detailed columns/types yet.

Entities:

- Books
- Members (students + librarians)
- Borrow (who borrowed what, when)
- Author
- Publisher
- Book Type / Category (Science, History, Fiction, ...)
- Fine / Invoice (part of Borrow or separate)

Logical data model

Detailed schema: columns, types, relationships (FKs).

Table	Fields
Books	<code>book_id</code> , <code>title</code> , <code>author_id</code> (FK), <code>publisher_id</code> (FK), <code>category_id</code> (FK)
Members	<code>member_id</code> , <code>name</code> , <code>email</code> , <code>address</code>
Borrow	<code>borrow_id</code> , <code>book_id</code> (FK), <code>member_id</code> (FK), <code>borrow_date</code> , <code>return_date</code>
Author	<code>author_id</code> , <code>name</code> , <code>address</code>
Publisher	<code>publisher_id</code> , <code>name</code> , <code>date_of_publication</code>
Book Type	<code>category_id</code> , <code>category_name</code>
Fine / Invoice	<code>fine_id</code> , <code>borrow_id</code> (FK), <code>amount</code>

Physical data model

How the logical model is **implemented** in a specific DBMS: `CREATE TABLE` , types, constraints, indexes, storage. Syntax is DBMS-specific (MySQL ≠ PostgreSQL in details).

```
Conceptual → Logical → Physical
(entities)  (schema)  (CREATE TABLE + indexes)
```

Types of DBMS

1. Relational DBMS (RDBMS)

Tables with rows and columns; relationships via PK/FK; predefined schema.

Examples: MySQL, PostgreSQL, Oracle, MS SQL Server, DB2, Sybase

2. Object-relational DBMS

Extends RDBMS for complex types: maps, audio, images, custom objects.

Real-estate example: search flats within 5 km of GPS location — geospatial types and spatial queries. Plain RDBMS does not do this natively.

3. NoSQL (Not Only SQL)

Not row/column oriented. Good for unstructured / semi-structured data with **varying schemas**.

MongoDB: JSON-like documents; each document in a collection can have different fields.

Instagram-style posts: caption / hashtags / tags optional — rigid relational tables need many nulls or awkward joins; document stores handle variable structure naturally.

SQL Language Categories

DQL — Data Query Language

Fetch / retrieve data.

Command	Purpose
SELECT	Retrieve rows from one or more tables

DML — Data Manipulation Language

Change **data** inside tables.

Command	Purpose
INSERT	Add new rows
UPDATE	Modify existing rows
DELETE	Remove rows

DDL — Data Definition Language

Define / change **structure**.

Command	Purpose
CREATE	Create database or table
ALTER	Modify existing table structure
DROP	Delete table or database entirely
TRUNCATE	Remove all rows; keep table structure

DCL — Data Control Language

Manage permissions.

Command	Purpose
GRANT	Give permission for specific operations
REVOKE	Remove a previously granted permission

Example idea: grant user `aniket` only `SELECT` on `student` — no writes.

TCL — Transaction Control Language

Manage transactions and consistency.

Command	Purpose
COMMIT	Permanently save changes in the current transaction
ROLLBACK	Undo changes since the last commit
SAVEPOINT	Checkpoint inside a transaction to roll back to

DQL → read DML → change rows
DDL → change structure DCL → permissions
TCL → commit / rollback

All SQL Commands from This Session

Commands actually run in class. Work through them in order at the `mysql>` prompt.

Check MySQL version (from CMD, before login)

```
mysql --version
```

```
mysql Ver 8.0.31 Distrib 8.0.31, for Win64 (x86_64)
```

Confirms install + PATH.

Log into MySQL

```
mysql -u root -p
```

Enter root password → `mysql>` prompt.

Show all databases

```
SHOW DATABASES;
```

```
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| sys |  
+-----+
```

These four are **default** — do not modify them casually.

Create a database

```
CREATE DATABASE august26;
```

Creates database `august26`. Names are effectively **case-insensitive** in MySQL on Windows.

Select a database

```
USE august26;
```

```
Database changed
```

All later commands apply to `august26` until you switch.

Create the `student` table (first version)

```
CREATE TABLE student (  
  roll_number INT,  
  name        VARCHAR(20),  
  age         INT,  
  email_id    VARCHAR(20)  
);
```

Four columns. `email_id VARCHAR(20)` is intentionally tight — it will error on a long email shortly.

Second table — case insensitivity demo

```
CREATE TABLE Student2 (  
  roll_number INT,  
  name        VARCHAR(20),  
  age         INT,  
  email_id    VARCHAR(20)  
);
```

Created as `Student2` ; MySQL typically stores/shows it as `student2` (lowercase). Table names are **case-insensitive** here.

Show all tables

```
SHOW TABLES;
```

```
+-----+  
| Tables_in_august26 |  
+-----+  
| student            |  
| student2           |  
+-----+
```

Describe a table structure

```
DESCRIBE student;  
-- or equivalently:  
DESC student;
```

Field	Type	Null	Key	Default	Extra
roll_number	int	YES		NULL	
name	varchar(20)	YES		NULL	
age	int	YES		NULL	
email_id	varchar(20)	YES		NULL	

Columns: name, type, NULL allowed?, key, default, extra. **Key** / **Extra** matter later for PKs and `AUTO_INCREMENT`.

Insert — error: data too long

```
INSERT INTO student (roll_number, name, age, email_id)
VALUES (1, 'Aniket', 30, 'aniket.mauri@gmail.com');
```

```
ERROR 1406 (22001): Data too long for column 'email_id' at row 1
```

`aniket.mauri@gmail.com` is longer than 20 characters.

Insert — success with a shorter email

```
INSERT INTO student (roll_number, name, age, email_id)
VALUES (1, 'Aniket', 30, 'aniket@gmail.com');
```

```
Query OK, 1 row affected
```

Lesson: read the error carefully — it names the column and the row.

Select all columns

```
SELECT * FROM student;
```

roll_number	name	age	email_id
1	Aniket	30	aniket@gmail.com

* = all columns. Prefer listing only needed columns in real systems (less data transfer).

Select specific columns

```
SELECT roll_number, name FROM student;
```

roll_number	name
1	Aniket

Create a table with a JSON column

```
CREATE TABLE books_management (  
  book_id INT,  
  book_info JSON  
);
```

MySQL can store JSON natively.

Describe books_management

```
DESCRIBE books_management;
```

Field	Type	Null	Key	Default	Extra
book_id	int	YES		NULL	
book_info	json	YES		NULL	

Insert a JSON document

```
INSERT INTO books_management (book_id, book_info)
VALUES (
  1,
  '{
    "library": {
      "name": "Central Library",
      "location": "Mumbai",
      "establishment": "1990",
      "books": [
        { "title": "Data Structures", "author": "Aniket" }
      ],
      "members": [
        { "name": "Yash", "email": "yash@gmail.com" }
      ]
    }
  }'
);
```

Query OK, 1 row affected

The whole object lives in one `book_info` cell.

Select all from `books_management`

```
SELECT * FROM books_management;
```

Returns `book_id = 1` and the full JSON (may wrap awkwardly in the terminal).

Query a field inside JSON

```
SELECT book_id,
       book_info → '$.library.name' AS library_name
FROM books_management;
```

+	+	+
	book_id	library_name
+	+	+
	1	"Central Library"
+	+	+

- `→` extracts a JSON value.
- `$.library.name` = start at root (`$`) → `library` → `name` .

Nested variants from class:

```
-- Book title from nested array (path as demonstrated)
SELECT book_id,
       book_info → '$.library.books[0].title' AS book_title
FROM books_management;

-- Member name
SELECT book_id,
       book_info → '$.library.members[0].name' AS member_name
FROM books_management;
```

JSON in MySQL is for awareness only — this module will not rely on it going forward.

Drop a table

```
DROP TABLE student;
```

Query OK, 0 rows affected

Removes the table (and its data) permanently. Structure is gone — unlike `TRUNCATE`, which keeps the empty table.

Quick revision checklist

Topic	One-line takeaway
Ports	MySQL 3306 , PostgreSQL 5432
Access	Prefer CMD: <code>mysql -u root -p</code>
DIKW	Data → Information → Knowledge → Wisdom
Data types	Structured / Unstructured / Semi-structured (JSON)
File system pain	Redundancy, inconsistency, isolation, integrity, security, locking, atomicity
RDBMS win	Queries, PK/FK, row locks, GRANT, transactions
CHAR vs VARCHAR	Fixed vs variable length
Index	Avoid full table scan for hot lookups
Models	Conceptual → Logical → Physical
SQL families	DQL, DML, DDL, DCL, TCL
First DDL/DML	<code>CREATE DATABASE</code> , <code>USE</code> , <code>CREATE TABLE</code> , <code>INSERT</code> , <code>SELECT</code> , <code>DROP</code>
Common error	<code>Data too long for column</code> → widen <code>VARCHAR</code>

Coverage: DBT Module Day 1 (08 Oct 2026) — install, DIKW, data types, file system vs RDBMS, keys, CHAR/VARCHAR, indexes, three data models, DBMS types, SQL categories, and every query run in the session.